

Hybrid Flow Architecture: Structured + NLP Assessment

Research Document 05 — Modality-Switching Design for the Civic Blueprint Assessment

Date: 2026-03-09 **Status:** Design specification **Depends on:** Research 02 (adaptive sequencing), Research 03 (conversation router), Research 06 (entropy confidence) **Integrates with:** `src/types/conversation.ts`, `src/stores/userStore.ts`, `src/lib/ballotHelpers.ts`

Table of Contents

- 1. System Context
 - 2. Routing Trigger Taxonomy
 - 3. Modality-Switch UX Contract
 - 4. Unified Session State
 - 5. Multi-Axis Extraction from Voice Input
 - 6. Conflict Resolution
 - 7. Adaptive Sequencing Integration
 - 8. End-to-End Walkthrough
 - 9. Implementation Notes
-

1. System Context

The Two Modalities

Structured path (primary). The user sees one question per axis, presented as 5 selectable cards. Each card corresponds to a discrete position on the 0-10 scale: `[0.0, 2.5, 5.0, 7.5, 10.0]` (mapped from normalized `[-0.85, -0.45, 0.0, +0.45, +0.85]`). Selection produces a `UserValueRecord` with a deterministic `confidence_assigned` based on position extremity.

NLP path (secondary / escape hatch). On any structured question, the user can tap "None of these fit -- tell us in your own words." This opens a voice or text input panel. The LLM interprets the free-form input using Template A (two-pass extraction) and produces one or more `ValueSignal` records, which are converted to `UserValueRecord` format via the entropy-confidence pipeline from Research 06.

Key architectural constraint: Both paths must produce `UserValueRecord` objects that are interchangeable in the matching formula. The downstream scoring engine (`computeCandidateMatches`) must not know or care which modality produced a given record.

The 16 Axes

The full axis set from Research 02 applies:

Index	Axis ID	Domain
-------	---------	--------

Index	Axis ID	Domain
0	econ_safetynet	Economic
1	econ_investment	Economic
2	econ_school_choice	Economic
3	econ_tax_structure	Economic
4	health_coverage_model	Healthcare
5	health_cost_control	Healthcare
6	health_public_health	Healthcare
7	housing_supply_zoning	Housing
8	housing_affordability_tools	Housing
9	housing_transport_priority	Housing
10	justice_policing_accountability	Justice
11	justice_sentencing_goals	Justice
12	justice_firearms	Justice
13	climate_ambition	Climate
14	climate_energy_portfolio	Climate
15	climate_permitting	Climate

Design Goals

1. A user who answers all 16 questions via cards gets the same quality profile as the current structured-only path.
2. A user who uses voice on some questions gets at least equivalent quality, and potentially completes faster because voice responses can score multiple axes simultaneously.
3. Modality switching feels natural, not jarring. The system adapts to the user, not the reverse.
4. The assessment can complete in as few as 8-10 interactions (structured + voice combined) instead of the full 16-question battery, when voice responses are information-rich.

2. Routing Trigger Taxonomy

2.1 Trigger Categories

Every modality switch is initiated by exactly one trigger. Triggers are classified by source and direction.

```
type ModalitySwitchDirection = 'structured_to_nlp' | 'nlp_to_structured';

type TriggerSource =
  | 'user_initiated'
```

```

| 'confidence_triggered'
| 'axis_type_triggered'
| 'engagement_triggered';

interface ModalityTrigger {
  id: string;
  direction: ModalitySwitchDirection;
  source: TriggerSource;
  /** The measurable signal that fires this trigger */
  signal: string;
  /** Threshold or condition for the signal */
  condition: string;
  /** What the system does when the trigger fires */
  action: string;
  /** Whether the user can override (dismiss) this trigger */
  dismissable: boolean;
}

```

2.2 Structured-to-NLP Triggers

T1: User taps "None of these fit" (user_initiated)

```

Signal:    user_action === 'escape_hatch_tap'
Condition: Always fires on tap
Action:    Expand NLP input panel below the cards. Cards remain visible
           but grayed out. Focus moves to the text/voice input area.
           Show the prompt: "Tell us what you think about [axis topic]
           in your own words."
Dismissable: N/A (user-initiated)

```

This is the primary entry point. The user sees 5 cards, none feel right, and taps the escape hatch. The NLP panel opens inline -- not a full page transition. Cards stay visible as context.

T2: Neutral selection with low importance (confidence_triggered)

```

Signal:    selected_position === 5.0 AND dwell_time_ms < 2000
           AND axis.importance_self_report < 4 (if available)
Condition: User picked the middle card quickly, suggesting confusion
           rather than genuine centrism
Action:    After the selection is registered, show a gentle prompt:
           "That's a nuanced topic. Want to explain your thinking?
           Sometimes the middle ground means different things to
           different people."
           Show a small "Tell us more" chip below the selected card.
Dismissable: Yes. If dismissed, the neutral selection stands with
           the confused-neutral sigma (3.9) from Research 06.

```

This trigger fires post-selection, not pre-selection. It does not block the user from proceeding; it offers an optional refinement. If the user engages, the NLP response replaces the low-confidence neutral record with a higher-quality signal.

T3: Repeated neutral selections (engagement_triggered)

```
Signal:    consecutive_neutrals >= 2
Condition: User has picked the middle card for 2+ consecutive axes
Action:    Show a mode-switch prompt: "It seems like the card options
           aren't capturing your views well. Would you rather just
           talk through the remaining topics? You can always switch
           back to cards."
           Offer a toggle: "Switch to conversation mode"
Dismissable: Yes. If dismissed, continue with cards.
```

Two consecutive neutrals strongly suggest either fatigue or misfit with the card format. The system offers a wholesale mode switch, not just per-question escape.

T4: Axis with known card-format weakness (axis_type_triggered)

```
Signal:    axis_id IN NUANCED_AXES
Condition: The axis is one where the 5-card format is known to
           compress too much nuance. Currently identified:
           - housing_supply_zoning (YIMBY/NIMBY is context-dependent)
           - climate_permitting (tradeoffs are multi-dimensional)
           - justice_policing_accountability (position depends on
             specific reform proposals)
Action:    Show the NLP escape hatch with enhanced prominence --
           slightly larger, with helper text: "This topic has a lot
           of nuance. The cards capture the basics, but feel free to
           explain your view if none quite fits."
Dismissable: N/A (it's just enhanced visual prominence, not a modal)
```

```
const NUANCED_AXES: Set<string> = new Set([
  'housing_supply_zoning',
  'climate_permitting',
  'justice_policing_accountability',
]);
```

This is a soft trigger -- it does not force a modality switch, just makes the NLP option more visually prominent for axes where card-based answers are known to lose fidelity.

2.3 NLP-to-Structured Triggers

T5: User requests cards back (user_initiated)

Signal: `user_action === 'show_cards' OR user_text matches intent 'want_to_see_options'`

Condition: User explicitly asks for the card options

Action: Collapse the NLP panel. Re-display cards for the current axis, highlighting any card that aligns with what the user already said (if a signal was partially extracted). Show: "Here are the options -- pick the one closest to what you described, or keep telling us more."

Dismissable: N/A (user-initiated)

T6: Vague or off-topic NLP response (confidence_triggered)

Signal: `nlp_extraction_result.primary_signal.confidence < 0.30 AND nlp_extraction_result.primary_signal.direction is within [3.5, 6.5]`

Condition: LLM could not extract a clear directional signal from the voice/text input

Action: Show the cards with the message: "Thanks for sharing that. To make sure we capture your view accurately, could you pick the card that feels closest? You can also try rephrasing if you'd like."

Dismissable: Yes. User can try voice again or pick a card.

T7: Multi-turn NLP without convergence (engagement_triggered)

Signal: `nlp_turns_on_current_axis >= 3 AND axis.confidence < 0.50`

Condition: User has spoken/typed 3 times about this axis but we still can't resolve their position

Action: Transition to cards: "I want to make sure I'm getting this right. Looking at these options, which one is closest to your view?" Show cards with any partial signal highlighted.

Dismissable: Yes. User can choose to skip the axis entirely.

2.4 Trigger Priority

When multiple triggers could fire simultaneously, apply this precedence:

1. User-initiated triggers (T1, T5) always win -- never override the user's explicit choice.
2. Confidence-triggered (T2, T6) fire only if no user action was taken.
3. Engagement-triggered (T3, T7) fire only after a pattern is established (multiple interactions).
4. Axis-type-triggered (T4) is visual-only and always applies regardless of other triggers.

```

function evaluateTriggers(context: TriggerContext): ModalityTrigger | null {
  // Priority 1: User-initiated
  if (context.userAction === 'escape_hatch_tap') return TRIGGERS.T1;
  if (context.userAction === 'show_cards') return TRIGGERS.T5;

  // Priority 2: Confidence-triggered
  if (context.currentModality === 'structured') {
    if (isConfusedNeutral(context)) return TRIGGERS.T2;
  }
  if (context.currentModality === 'nlp') {
    if (isVagueNlpResponse(context)) return TRIGGERS.T6;
  }

  // Priority 3: Engagement-triggered
  if (context.consecutiveNeutrals >= 2) return TRIGGERS.T3;
  if (context.nlpTurnsOnCurrentAxis >= 3 && context.axisConfidence < 0.50) {
    return TRIGGERS.T7;
  }

  // No trigger fires
  return null;
}

function isConfusedNeutral(ctx: TriggerContext): boolean {
  return (
    ctx.selectedPosition === 5.0 &&
    ctx.dwellTimeMs < 2000 &&
    (ctx.importanceSelfReport === undefined || ctx.importanceSelfReport < 4)
  );
}

function isVagueNlpResponse(ctx: TriggerContext): boolean {
  return (
    ctx.nlpExtractionResult !== null &&
    ctx.nlpExtractionResult.primary_signal.confidence < 0.30 &&
    ctx.nlpExtractionResult.primary_signal.direction >= 3.5 &&
    ctx.nlpExtractionResult.primary_signal.direction <= 6.5
  );
}

```

2.5 Trigger Summary Table

ID	Direction	Source	Signal	Condition	Dismissable
T1	S -> NLP	user_initiated	Tap "none of these fit"	Always	No
T2	S -> NLP	confidence_triggered	Neutral + fast dwell	pos=5.0, dwell<2s	Yes
T3	S -> NLP	engagement_triggered	Repeated neutrals	2+ consecutive	Yes

ID	Direction	Source	Signal	Condition	Dismissable
T4	S -> NLP	axis_type_triggered	Nuanced axis	axis in NUANCED_AXES	N/A (visual)
T5	NLP -> S	user_initiated	User requests cards	Explicit action	No
T6	NLP -> S	confidence_triggered	Vague NLP extraction	conf<0.30, dir~5.0	Yes
T7	NLP -> S	engagement_triggered	No convergence	3+ turns, conf<0.50	Yes

3. Modality-Switch UX Contract

3.1 Transition Principles

- Transitions must acknowledge what the user just did, not discard it.
- Transitions should feel like the system is adapting to the user, not correcting them.
- The new modality panel should appear inline, not as a full-page navigation. Context from the previous modality remains visible.
- Every transition phrase references the topic in plain language, not axis IDs.

3.2 Structured-to-NLP Transitions

When user taps "None of these fit" (T1):

"Got it -- those options don't quite capture your view on [housing affordability]. Go ahead and tell us what you think, in whatever way feels natural."

"No worries. [School choice] is a topic where people's views don't always fit neatly into boxes. What's your take?"

"Sure thing. Just share your thoughts on [gun policy] -- a sentence or two is plenty, but say as much as you'd like."

When system detects confused neutral (T2):

"That's a topic where 'it depends' is a totally valid answer. If you'd like, you can tell us what it depends on -- that helps us match you more precisely."

"The middle ground on [healthcare costs] can mean a lot of different things. Want to tell us what it means to you?"

"Picking the balanced option is fine. But if there's nuance we're missing, you can always tell us more."

When system detects fatigue / repeated neutrals (T3):

"It seems like the card options aren't quite landing. Want to switch to a more conversational format? You can just talk through the remaining topics."

"We can also do this as a quick conversation instead of cards -- some people find that easier. Want to try it?"

"You've been picking the middle a few times. If that's because none of the options feel right, we can switch to free-form -- just say what matters to you."

3.3 NLP-to-Structured Transitions

When user requests cards (T5):

"Sure -- here are the options for [climate ambition]. If what you told us matches one of these, just tap it to confirm."

"Here are the five positions. Based on what you said, [this one] seems closest -- but pick whichever feels right."

"Switching back to cards. Take a look and see which one fits best."

When NLP response is vague (T6):

"Thanks for sharing that. To make sure we capture your view accurately, take a look at these options -- which one feels closest?"

"I appreciate the nuance. Sometimes it helps to pick a card as a starting point, even if it's not perfect. Which is closest?"

"That's a complex topic. Let me show you the options -- picking one helps us understand where you land, even approximately."

When NLP doesn't converge after multiple turns (T7):

"I want to make sure I'm getting this right. Let me show you the options for [sentencing policy] -- sometimes seeing them laid out helps."

"We've gone back and forth a bit on this one. Looking at these five options, which resonates most? You can also skip this one entirely."

"Let me try a different approach -- here are the main positions on [transit vs. roads]. Pick the closest fit, or skip if none works."

4. Unified Session State

4.1 Core State Extension

The existing `ConversationSession` from `src/types/conversation.ts` tracks the warmup conversation and ballot-item discussions. The hybrid flow introduces a new `HybridAssessmentSession` that wraps the adaptive sequencer (Research 02) and conversation router (Research 03) into a single session with per-axis modality tracking.

```
// =====  
// Modality tracking per axis  
// =====  
  
type InputModality = 'structured' | 'nlp' | 'imputed';
```



```

interface ModalityRecord {
    /** Which modality produced this axis's current best record */
    modality: InputModality;
    /** If NLP, how many voice/text turns contributed */
    nlp_turn_count: number;
    /** If structured, the raw card selection (0-10 scale) */
    structured_selection?: number;
    /** If the user switched modalities on this axis, log the history */
    switch_history: ModalitySwitch[];
}

interface ModalitySwitch {
    from: InputModality;
    to: InputModality;
    trigger_id: string;          // T1-T7
    turn_number: number;
    /** Confidence at the time of switch (for the 'from' modality) */
    confidence_at_switch: number;
}

// =====
// Per-axis confidence by modality
// =====

interface AxisConfidenceByModality {
    /** Confidence from the structured card selection, if any */
    structured_confidence: number | null;
    /** Confidence from NLP extraction, if any */
    nlp_confidence: number | null;
    /** Confidence from cross-axis imputation, if any */
    imputed_confidence: number | null;
    /** The merged/final confidence used in the matching formula */
    merged_confidence: number;
}

// =====
// Signal classification for multi-axis NLP extraction
// =====

type SignalStrength = 'primary' | 'secondary' | 'spillover';

interface ClassifiedSignal {
    axis_id: string;
    strength: SignalStrength;
    /** ValueSignal from LLM extraction */
    raw_signal: ValueSignal;
    /** Entropy-based confidence after processing */
    entropy_confidence: number;
    /** Hybrid confidence (entropy + heuristic blend from Research 06) */
    hybrid_confidence: number;
    /** Whether the user has confirmed this signal */
    user_confirmed: boolean;
}

```

```

// =====
// Full hybrid session state
// =====

interface HybridAssessmentSession {
    session_id: string;

    // --- Global session fields ---
    /** Total user interactions (card selections + NLP turns) */
    interaction_count: number;
    /** Maximum allowed interactions before forced completion */
    max_interactions: number; // default: 20
    /** Current modality preference (what the user sees next by default) */
    default_modality: 'structured' | 'nlp';

    // --- Per-axis state ---
    axes: Record<string, HybridAxisState>;

    // --- Sequencing ---
    /** Ordered list of axes to present (from adaptive sequencer) */
    question_queue: string[];
    /** Index into question_queue */
    current_question_index: number;
    /** Axes that have been answered (directly or via multi-axis NLP) */
    answered_axes: Set<string>;
    /** Axes confirmed by user (subset of answered_axes) */
    confirmed_axes: Set<string>;
    /** Axes skipped explicitly by the user */
    skipped_axes: Set<string>;

    // --- Modality history ---
    /** Chronological log of all modality switches */
    modality_switch_log: ModalitySwitch[];
    /** How many times the user has used NLP (for engagement tracking) */
    total_nlp_interactions: number;
    /** How many times the user has used structured (for engagement tracking) */
    total_structured_interactions: number;
    /** Running count of consecutive neutral structured selections */
    consecutive_neutrals: number;

    // --- Pending multi-axis signals ---
    /** Secondary signals from NLP that await user confirmation */
    pending_secondary_signals: ClassifiedSignal[];

    // --- Session-level entropy (from Research 03) ---
    session_entropy: number;
    estimated_remaining_interactions: number;
    ready_for_matching: boolean;
}

interface HybridAxisState {
    axis_id: string;

```

```

// --- Posterior (from Research 02/03) ---
/** Discretized posterior. 5-point for structured sequencer, 11-point
 * for NLP router -- the hybrid uses the 21-bin version from Research 06
 * as the unified representation. */
posterior: number[]; // length 21, bins at 0.0, 0.5, 1.0, ..., 10.0
entropy: number;
point_estimate: number;

// --- Modality tracking ---
modality_record: ModalityRecord;
confidence_by_modality: AxisConfidenceByModality;

// --- Records ---
/** The current best UserValueRecord for this axis */
current_record: UserValueRecord | null;
/** All signals ever received for this axis (audit trail) */
signal_history: ClassifiedSignal[];

// --- Coverage ---
coverage_status: 'uncovered' | 'soft_only' | 'partial' | 'covered';
importance: number | null;

// --- Evidence ---
evidence_quotes: string[];
}

```

4.2 Unified UserValueRecord

Both modalities produce records in this format. The `source_modality` field is informational only -- the matching formula ignores it.

```

interface UserValueRecord {
  axis_id: string;
  /** Position on the 0-10 scale */
  score: number;
  /** Normalized position on the -1 to +1 scale (for matching formula) */
  score_normalized: number;
  /** Entropy-hybrid confidence (0-1) */
  confidence: number;
  /** Coverage status for the matching formula */
  coverage_status: 'answered' | 'imputed' | 'uncovered';
  is_imputed: boolean;
  imputation_source?: string[];

  // --- Provenance (informational, not used in matching) ---
  source_modality: InputModality;
  /** If structured: which card position was selected */
  card_position?: number;
  /** If NLP: the user's verbatim input */
  nlp_source_text?: string;
  /** If NLP: the LLM's reasoning chain */
}

```

```

nlp_reasoning?: string;
/** If NLP: which signal strength classification */
signal_strength?: SignalStrength;
/** Timestamp of the record */
recorded_at: number;
}

```

4.3 Session Initialization

```

function initializeHybridSession(
  session_id: string,
  ballot_relevant_axes: string[],
  returning_user_profile?: Record<string, UserValueRecord>
): HybridAssessmentSession {
  const axes: Record<string, HybridAxisState> = {};

  for (const axis_id of ALL_AXIS_IDS) {
    const returning_record = returning_user_profile?.[axis_id] ?? null;
    const has_prior = returning_record !== null;

    axes[axis_id] = {
      axis_id,
      posterior: has_prior
        ? buildPosteriorFromRecord(returning_record)
        : uniformPosterior21(),
      entropy: has_prior ?
computeEntropy21(buildPosteriorFromRecord(returning_record)) : H_MAX_21,
      point_estimate: has_prior ? returning_record.score : 5.0,
      modality_record: {
        modality: has_prior ? returning_record.source_modality : 'structured',
        nlp_turn_count: 0,
        switch_history: [],
      },
      confidence_by_modality: {
        structured_confidence: has_prior && returning_record.source_modality ===
'structured'
          ? returning_record.confidence : null,
        nlp_confidence: has_prior && returning_record.source_modality === 'nlp'
          ? returning_record.confidence : null,
        imputed_confidence: null,
        merged_confidence: has_prior ? returning_record.confidence : 0,
      },
      current_record: returning_record,
      signal_history: [],
      coverage_status: has_prior ? 'covered' : 'uncovered',
      importance: null,
      evidence_quotes: [],
    };
  }

  // Initial question ordering from the adaptive sequencer

```

```

const initial_queue = computeInitialQuestionOrder(axes, ballot_relevant_axes);

return {
  session_id,
  interaction_count: 0,
  max_interactions: 20,
  default_modality: 'structured',
  axes,
  question_queue: initial_queue,
  current_question_index: 0,
  answered_axes: new Set(
    returning_user_profile ? Object.keys(returning_user_profile) : []
  ),
  confirmed_axes: new Set(
    returning_user_profile ? Object.keys(returning_user_profile) : []
  ),
  skipped_axes: new Set(),
  modality_switch_log: [],
  total_nlp_interactions: 0,
  total_structured_interactions: 0,
  consecutive_neutrals: 0,
  pending_secondary_signals: [],
  session_entropy: Object.values(axes).reduce((sum, a) => sum + a.entropy, 0),
  estimated_remaining_interactions: initial_queue.length,
  ready_for_matching: false,
};
}

```

5. Multi-Axis Extraction from Voice Input

5.1 The Multi-Axis Opportunity

When a user provides voice/text input for axis X, their response frequently contains signals for other axes. Example:

Asked about: housing affordability tools **User says:** "I think the government should build more public housing and invest in public transit so people don't need cars to get to work. Rent control is a band-aid -- we need to actually build more units and loosen zoning restrictions."

Signals present:

- **housing_affordability_tools** (primary): government subsidies, score ~2.0, high confidence
- **housing_transport_priority** (secondary): public transit investment, score ~2.0, moderate confidence
- **housing_supply_zoning** (secondary): loosen zoning, score ~2.5, moderate confidence

Without multi-axis extraction, only the affordability signal is captured. The user would still be asked about transit and zoning separately. With multi-axis extraction, all three housing axes can be resolved in a single interaction.

5.2 Signal Classification

```
interface MultiAxisExtractionResult {
  /** The axis the question was about -- always gets primary classification */
  asked_axis: string;
  /** All extracted signals, classified by strength */
  signals: ClassifiedSignal[];
  /** Summary for the user confirmation UI */
  extraction_summary: ExtractionSummary;
}

interface ExtractionSummary {
  primary: { axis_id: string; position_label: string; confidence: number };
  secondary: Array<{ axis_id: string; position_label: string; confidence: number
}>;
  /** Plain-language summary for the user */
  summary_text: string;
}
```

Classification rules:

```
function classifySignals(
  asked_axis: string,
  raw_signals: ValueSignal[]
): ClassifiedSignal[] {
  return raw_signals.map(signal => {
    let strength: SignalStrength;

    if (signal.axisId === asked_axis) {
      // Primary: the axis we asked about
      strength = 'primary';
    } else if (signal.confidence >= 0.50 && isExplicitMention(signal)) {
      // Secondary: user explicitly mentioned this topic with sufficient clarity
      strength = 'secondary';
    } else {
      // Spillover: inferred from correlation, not directly stated
      strength = 'spillover';
    }

    // Compute entropy-hybrid confidence
    const entropy_result = entropyConfidenceNLP(signal.direction,
signal.confidence);
    const hybrid_conf = computeHybridConfidence(entropy_result.confidence,
signal.confidence);

    // Apply strength-based confidence discount
    const discounted_conf = applyStrengthDiscount(hybrid_conf, strength);

    return {
      axis_id: signal.axisId,
```

```

    strength,
    raw_signal: signal,
    entropy_confidence: entropy_result.confidence,
    hybrid_confidence: discounted_conf,
    user_confirmed: strength === 'primary', // primary is auto-confirmed
  });
});
}

/**
 * Check if the signal comes from an explicit user statement (not just
 * statistical spillover). The LLM extraction includes a `source` quote;
 * if the source quote directly references the axis topic, it's explicit.
 */
function isExplicitMention(signal: ValueSignal): boolean {
  // The source field contains the user's words that triggered this signal.
  // If it's a direct quote (not a paraphrase from the spillover graph),
  // the signal is explicit.
  return signal.source.length > 10 && !signal.source.startsWith('[implied]');
}

/**
 * Strength-based confidence discount.
 *
 * Primary signals use full confidence.
 * Secondary signals are discounted by 15% -- the user mentioned this
 * topic but we weren't specifically asking, so there's more
 * interpretation risk.
 * Spillover signals are discounted by 40% -- these are statistical
 * inferences, not direct statements.
 */
function applyStrengthDiscount(confidence: number, strength: SignalStrength):
number {
  switch (strength) {
    case 'primary': return confidence;
    case 'secondary': return confidence * 0.85;
    case 'spillover': return confidence * 0.60;
  }
}

```

5.3 User Confirmation Flow

When secondary signals are extracted, the user sees a confirmation card:

```

+-----+
| We picked up a few things from your response: |
|
| [v] Housing affordability: Government subsidies | <- primary, auto-checked
| [ ] Public transit: More investment             | <- secondary, unchecked
| [ ] Zoning: Loosen restrictions                 | <- secondary, unchecked
|

```

```

| Check the ones that sound right, and we'll skip |
| those questions later.                         |
|                                                 |
| [Confirm & Continue]                         |
+-----+

```

Behavior:

- Primary signal: pre-checked, cannot be unchecked (the user explicitly answered this one).
- Secondary signals: unchecked by default. The user must opt in.
- Spillover signals: not shown to the user. They update the posterior as soft evidence but do not count as "answered" and do not skip future questions.

```

interface ConfirmationCard {
  primary: {
    axis_id: string;
    position_label: string;
    score: number;
    locked: true; // cannot be unchecked
  };
  secondary: Array<{
    axis_id: string;
    position_label: string;
    score: number;
    checked: boolean; // user can toggle
  }>;
}

function processConfirmation(
  session: HybridAssessmentSession,
  confirmed_axis_ids: string[], // axes the user checked
  all_signals: ClassifiedSignal[]
): HybridAssessmentSession {
  const updated = structuredClone(session);

  for (const signal of all_signals) {
    const axis = updated.axes[signal.axis_id];
    if (!axis) continue;

    if (signal.strength === 'primary') {
      // Always apply primary signals
      applySignalToAxis(axis, signal, true);
      updated.answered_axes.add(signal.axis_id);
      updated.confirmed_axes.add(signal.axis_id);
    } else if (signal.strength === 'secondary') {
      if (confirmed_axis_ids.includes(signal.axis_id)) {
        // User confirmed this secondary signal
        signal.user_confirmed = true;
        applySignalToAxis(axis, signal, true);
        updated.answered_axes.add(signal.axis_id);
        updated.confirmed_axes.add(signal.axis_id);
      }
    }
  }

  return updated;
}

```



```

    } else {
        // User did not confirm -- treat as soft evidence only
        applySignalToAxis(axis, signal, false);
        // Do NOT add to answered_axes -- the question will still be asked
    }
} else {
    // Spillover: always soft evidence
    applySignalToAxis(axis, signal, false);
}
}

// Recompute question queue (confirmed axes get skipped)
updated.question_queue = recomputeQuestionQueue(updated);
updated.session_entropy = computeSessionEntropy(updated);

return updated;
}

function applySignalToAxis(
    axis: HybridAxisState,
    signal: ClassifiedSignal,
    is_confirmed: boolean
): void {
    // Update posterior using the Bayesian update from Research 03
    const sigma = llmConfidenceToSigma(signal.hybrid_confidence);
    axis.posterior = bayesianUpdatePosterior21(
        axis.posterior,
        signal.raw_signal.direction,
        sigma,
        signal.strength === 'spillover' // spillover gets flattened likelihood
    );
    axis.entropy = computeEntropy21(axis.posterior);
    axis.point_estimate = computeExpectedValue21(axis.posterior);

    // Update confidence tracking
    axis.confidence_by_modality.nlp_confidence = signal.hybrid_confidence;
    axis.confidence_by_modality.merged_confidence = signal.hybrid_confidence;

    // Update coverage
    if (is_confirmed && signal.hybrid_confidence >= 0.50) {
        axis.coverage_status = 'covered';
    } else if (signal.hybrid_confidence >= 0.25) {
        axis.coverage_status = axis.coverage_status === 'uncovered' ? 'partial' :
axis.coverage_status;
    } else {
        axis.coverage_status = axis.coverage_status === 'uncovered' ? 'soft_only' :
axis.coverage_status;
    }

    // Build UserValueRecord if confirmed
    if (is_confirmed) {
        axis.current_record = {
            axis_id: axis.axis_id,
            score: signal.raw_signal.direction,

```

```

        score_normalized: (signal.raw_signal.direction - 5.0) / 5.0,
        confidence: signal.hybrid_confidence,
        coverage_status: 'answered',
        is_imputed: false,
        source_modality: 'nlp',
        nlp_source_text: signal.raw_signal.source,
        nlp_reasoning: signal.raw_signal.reasoning,
        signal_strength: signal.strength,
        recorded_at: Date.now(),
    };
}

// Update modality record
axis.modality_record.modality = 'nlp';
axis.modality_record.nlp_turn_count++;

// Store signal in history
axis.signal_history.push(signal);

// Store evidence quote
if (signal.raw_signal.source) {
    axis.evidence_quotes.push(signal.raw_signal.source);
}
}

```

5.4 Extraction Prompt Design

The multi-axis extraction uses a modified Template A that scans all 16 axes, not just the current domain. This is the Phase 2b scan from Research 03 Section 3.

System prompt addition for multi-axis extraction:

The user was asked about {asked_axis_topic}. Extract their position on that axis as the PRIMARY signal. Then scan for any ADDITIONAL positions they expressed about other topics:

```

{for each axis not yet covered}
- {axis_id}: {axis_topic_hint} (look for: {pole_a_keywords} vs {pole_b_keywords})
{end for}

```

For each additional axis where the user made a clear, explicit statement:

- Include it as a SECONDARY signal with the source quote
- Only include axes where the user DIRECTLY stated a position (not inferred)
- Set confidence based on how explicit and unambiguous the statement was

Do NOT include axes where the connection is only statistical/correlational. Those are handled by the spillover graph, not the LLM.

6. Conflict Resolution

6.1 When Conflicts Arise

An axis can receive scores from both modalities when:

1. User selects a card, then later provides a voice response about the same axis (via a different question's secondary signal).
2. User provides a voice response, is dissatisfied, and switches to cards.
3. The adaptive sequencer revisits an axis because new information changed the question queue.

6.2 Merge Rule

```
interface MergeInput {
  structured: UserValueRecord | null;
  nlp: UserValueRecord | null;
}

/**
 * Merge axis records from structured and NLP modalities.
 *
 * The merge rule follows three principles:
 * 1. Recency: The most recent record gets a temporal bonus.
 * 2. Confidence: Higher confidence records contribute more.
 * 3. User agency: If the user explicitly switched modalities (trigger T1 or T5),
 *    the destination modality's record takes full precedence.
 */
function mergeAxisRecords(
  input: MergeInput,
  switch_history: ModalitySwitch[]
): UserValueRecord {
  const { structured, nlp } = input;

  // Case 1: Only one modality has a record
  if (!structured && nlp) return nlp;
  if (structured && !nlp) return structured;
  if (!structured && !nlp) {
    throw new Error('Cannot merge: no records for either modality');
  }

  // Both records exist
  const s = structured!;
  const n = nlp!;

  // Case 2: User explicitly switched TO one modality (T1 or T5)
  // The destination modality takes full precedence -- the user
  // decided the other modality wasn't capturing their view.
  const last_switch = switch_history.length > 0
    ? switch_history[switch_history.length - 1]
    : null;

  if (last_switch) {
    if (last_switch.trigger_id === 'T1' && last_switch.to === 'nlp') {
      // User switched away from cards -- NLP wins
    }
  }
}
```

```

    return n;
  }
  if (last_switch.trigger_id === 'T5' && last_switch.to === 'structured') {
    // User switched back to cards -- structured wins
    return s;
  }
}

// Case 3: No explicit user-driven switch. Merge by confidence-weighted
// average, with a temporal bonus for the more recent record.
const recency_bonus = 0.05; // 5% bonus for the more recent record
const s_weight = s.confidence + (s.recorded_at > n.recorded_at ? recency_bonus :
0);
const n_weight = n.confidence + (n.recorded_at > s.recorded_at ? recency_bonus :
0);
const total_weight = s_weight + n_weight;

if (total_weight === 0) {
  // Both have zero confidence -- use whichever is more recent
  return s.recorded_at > n.recorded_at ? s : n;
}

// Weighted average of scores
const merged_score = (s.score * s_weight + n.score * n_weight) / total_weight;
const merged_score_normalized = (merged_score - 5.0) / 5.0;

// Confidence: take the maximum (the more informative record dominates)
// rather than averaging (which would dilute a high-confidence record
// with a low-confidence one).
const merged_confidence = Math.max(s.confidence, n.confidence);

// If the two scores are far apart (>3.0 on the 0-10 scale), this is a
// contradiction. Use the bimodal posterior from Research 06 and reduce
// confidence to reflect genuine uncertainty.
const score_gap = Math.abs(s.score - n.score);
const is_contradictory = score_gap > 3.0;

const final_confidence = is_contradictory
  ? Math.min(merged_confidence * 0.5, 0.40) // cap at 0.40 for contradictions
  : merged_confidence;

return {
  axis_id: s.axis_id,
  score: merged_score,
  score_normalized: merged_score_normalized,
  confidence: final_confidence,
  coverage_status: 'answered',
  is_imputed: false,
  source_modality: final_confidence === s.confidence ? 'structured' : 'nlp',
  card_position: s.card_position,
  nlp_source_text: n.nlp_source_text,
  nlp_reasoning: n.nlp_reasoning,
  recorded_at: Math.max(s.recorded_at, n.recorded_at),

```

```
};  
}
```

6.3 Contradiction Handling

When the structured and NLP scores diverge by more than 3.0 points:

1. The system does NOT silently merge. It shows the user the discrepancy.
2. The UI presents both positions side by side with a prompt:

"Earlier you picked [moderate government subsidies] for housing affordability, but you also said [the market should handle it]. These seem a bit different -- which is closer to where you stand?"

3. The user can:
 - Pick one position (the other is discarded).
 - Say "it's complicated" (the bimodal posterior from Research 06 is used, and the axis gets lower confidence).
 - Provide a new voice/text response that supersedes both.

```
interface ContradictionResolution {  
  strategy: 'pick_structured' | 'pick_nlp' | 'bimodal' | 'new_response';  
  /** If 'new_response', the new NLP input text */  
  new_input?: string;  
}  
  
function resolveContradiction(  
  axis: HybridAxisState,  
  structured_record: UserValueRecord,  
  nlp_record: UserValueRecord,  
  resolution: ContradictionResolution  
): UserValueRecord {  
  switch (resolution.strategy) {  
    case 'pick_structured':  
      return structured_record;  
  
    case 'pick_nlp':  
      return nlp_record;  
  
    case 'bimodal': {  
      // Use bimodal posterior from Research 06  
      const posterior = bimodalPosterior(  
        { direction: structured_record.score, confidence:  
structured_record.confidence },  
        { direction: nlp_record.score, confidence: nlp_record.confidence }  
      );  
      const entropy_result = entropyConfidenceFromPosterior(posterior);  
      return {  
        axis_id: axis.axis_id,  
        score: entropy_result.posteriorMean,  
        score_normalized: (entropy_result.posteriorMean - 5.0) / 5.0,  
        confidence: entropy_result.confidence,  

```

```

        coverage_status: 'answered',
        is_imputed: false,
        source_modality: 'nlp', // bimodal is a hybrid result
        recorded_at: Date.now(),
    };
}

case 'new_response':
    // This triggers a fresh NLP extraction pass. The caller
    // should process the new input through the standard NLP
    // pipeline, which will produce a new UserValueRecord that
    // supersedes both prior records.
    throw new Error('new_response must be handled by the caller via NLP
extraction');
}
}

```

7. Adaptive Sequencing Integration

7.1 How the Hybrid Flow Changes the Sequencer

The adaptive sequencer from Research 02 was designed for a pure structured path: one question per axis, 5 possible responses, deterministic confidence. The hybrid flow introduces three changes:

1. **Multi-axis voice responses remove axes from the queue.** When the user confirms secondary signals, those axes are marked as answered. The sequencer must recompute the next-best-question from the reduced set.
2. **NLP signals have variable confidence.** Unlike structured selections (confidence in `[0.70, 0.90]`), NLP signals can have confidence anywhere in `[0.10, 0.95]`. The posterior update must accept continuous confidence values, not just the 5-point structured map.
3. **Soft evidence from unconfirmed secondary signals and spillover changes the posterior landscape.** Even if an axis is not yet "answered," its posterior may no longer be uniform. The EWIG computation in the sequencer already handles this naturally -- reduced entropy means reduced expected information gain, which lowers the axis's priority.

7.2 Unified Posterior Update

The hybrid session maintains posteriors in the 21-bin format from Research 06 (bins at 0.0, 0.5, 1.0, ..., 10.0). Both structured and NLP signals update the same posterior.

```

/**
 * Process a structured card selection into the hybrid session.
 */
function processStructuredSelection(
    session: HybridAssessmentSession,
    axis_id: string,
    selected_value: number, // 0, 2.5, 5.0, 7.5, or 10.0

```

```

    confidence_assigned: number, // 0.70-0.90
    dwell_time_ms: number
): HybridAssessmentSession {
    const updated = structuredClone(session);
    const axis = updated.axes[axis_id];

    // Compute entropy-hybrid confidence (from Research 06)
    const entropy_result = entropyConfidenceStructured(
        selected_value,
        axis.importance ?? undefined,
        dwell_time_ms
    );
    const hybrid_conf = ALPHA_STRUCTURED * entropy_result.confidence
        + (1 - ALPHA_STRUCTURED) * confidence_assigned;

    // Build posterior from the structured selection
    const sigma = structuredSigma(selected_value, dwell_time_ms);
    axis.posterior = bayesianUpdatePosterior21(
        axis.posterior,
        selected_value,
        sigma,
        false // not spillover
    );
    axis.entropy = computeEntropy21(axis.posterior);
    axis.point_estimate = computeExpectedValue21(axis.posterior);

    // Build UserValueRecord
    axis.current_record = {
        axis_id,
        score: selected_value,
        score_normalized: (selected_value - 5.0) / 5.0,
        confidence: hybrid_conf,
        coverage_status: 'answered',
        is_imputed: false,
        source_modality: 'structured',
        card_position: selected_value,
        recorded_at: Date.now(),
    };

    axis.modality_record.modality = 'structured';
    axis.modality_record.structured_selection = selected_value;
    axis.confidence_by_modality.structured_confidence = hybrid_conf;
    axis.confidence_by_modality.merged_confidence = hybrid_conf;
    axis.coverage_status = 'covered';

    updated.answered_axes.add(axis_id);
    updated.confirmed_axes.add(axis_id);
    updated.interaction_count++;
    updated.total_structured_interactions++;

    // Track consecutive neutrals for T3 trigger
    if (selected_value === 5.0) {
        updated.consecutive_neutrals++;
    } else {

```

```

    updated.consecutive_neutrals = 0;
  }

  // Propagate cross-axis correlations (from Research 02)
  propagateCrossAxisPosteriors(updated, axis_id);

  // Recompute question queue
  updated.question_queue = recomputeQuestionQueue(updated);
  updated.session_entropy = computeSessionEntropy(updated);
  updated.ready_for_matching = evaluateSessionStopping(updated).shouldStop;

  return updated;
}

/**
 * Process an NLP voice/text response into the hybrid session.
 * This handles multi-axis extraction, classification, and the
 * confirmation flow.
 */
async function processNlpResponse(
  session: HybridAssessmentSession,
  asked_axis: string,
  user_input: string
): Promise<{
  updated_session: HybridAssessmentSession;
  confirmation_card: ConfirmationCard | null;
}> {
  // Step 1: Run LLM extraction (Template A, two-pass, all 16 axes)
  const raw_signals = await extractValueSignals(user_input, asked_axis, session);

  // Step 2: Classify signals by strength
  const classified = classifySignals(asked_axis, raw_signals);

  // Step 3: Apply primary signal immediately
  const updated = structuredClone(session);
  const primary_signals = classified.filter(s => s.strength === 'primary');
  const secondary_signals = classified.filter(s => s.strength === 'secondary');
  const spillover_signals = classified.filter(s => s.strength === 'spillover');

  // Apply primary (always confirmed)
  for (const signal of primary_signals) {
    applySignalToAxis(updated.axes[signal.axis_id], signal, true);
    updated.answered_axes.add(signal.axis_id);
    updated.confirmed_axes.add(signal.axis_id);
  }

  // Apply spillover as soft evidence (never shown to user)
  for (const signal of spillover_signals) {
    applySignalToAxis(updated.axes[signal.axis_id], signal, false);
  }

  // Step 4: Cross-axis propagation
  for (const signal of primary_signals) {
    propagateCrossAxisPosteriors(updated, signal.axis_id);
  }
}

```



```

}

updated.interaction_count++;
updated.total_nlp_interactions++;
updated.consecutive_neutrals = 0; // NLP input breaks the neutral streak

// Step 5: Build confirmation card if there are secondary signals
let confirmation_card: ConfirmationCard | null = null;
if (secondary_signals.length > 0) {
  updated.pending_secondary_signals = secondary_signals;

  confirmation_card = {
    primary: {
      axis_id: asked_axis,
      position_label: getPositionLabel(asked_axis,
primary_signals[0]?.raw_signal.direction ?? 5.0),
      score: primary_signals[0]?.raw_signal.direction ?? 5.0,
      locked: true,
    },
    secondary: secondary_signals.map(s => ({
      axis_id: s.axis_id,
      position_label: getPositionLabel(s.axis_id, s.raw_signal.direction),
      score: s.raw_signal.direction,
      checked: false,
    })),
  };
}

// Step 6: Recompute session state
updated.question_queue = recomputeQuestionQueue(updated);
updated.session_entropy = computeSessionEntropy(updated);
updated.ready_for_matching = evaluateSessionStopping(updated).shouldStop;

return { updated_session: updated, confirmation_card };
}

```

7.3 Question Queue Recomputation

After every interaction, the question queue is recomputed. The key change for the hybrid flow: axes that were answered via secondary NLP signals are removed from the queue.

```

function recomputeQuestionQueue(session: HybridAssessmentSession): string[] {
  const unanswered = ALL_AXIS_IDS.filter(
    id => !session.answered_axes.has(id) && !session.skipped_axes.has(id)
  );

  if (unanswered.length === 0) return [];

  // Check stopping criterion first
  const stopping = evaluateSessionStopping(session);
  if (stopping.shouldStop) return [];
}

```

```

// Sort by EWIG (Expected Weighted Information Gain) from Research 02
// The posteriors have already been updated with all signals (including
// soft evidence from unconfirmed secondaries and spillover), so the
// EWIG computation automatically accounts for the information already
// gathered via multi-axis NLP.
const scored = unanswered.map(axis_id => ({
  axis_id,
  ewig: computeEWIG(axis_id, session),
}));

// Apply domain diversity bonus (from Research 02)
const recent_domains = getRecentDomains(session, 2);
for (const item of scored) {
  const domain = axisToDomain(item.axis_id);
  if (recent_domains.length >= 2 && recent_domains.every(d => d ===
recent_domains[0])) {
    if (domain !== recent_domains[0]) {
      item.ewig *= 1.10;
    }
  }
}

scored.sort((a, b) => b.ewig - a.ewig);
return scored.map(s => s.axis_id);
}

function evaluateSessionStopping(session: HybridAssessmentSession): { shouldStop:
boolean; reason: string } {
  // Hard floor: at least 5 interactions (ensures domain coverage)
  if (session.interaction_count < 5) {
    return { shouldStop: false, reason: 'minimum interactions not reached' };
  }

  // Hard ceiling: max interactions
  if (session.interaction_count >= session.max_interactions) {
    return { shouldStop: true, reason: 'max interactions reached' };
  }

  // All axes answered or confirmed
  if (session.answered_axes.size >= ALL_AXIS_IDS.length) {
    return { shouldStop: true, reason: 'all axes answered' };
  }

  // Entropy floor: remaining weighted entropy < 20% of maximum
  const max_entropy = ALL_AXIS_IDS.length * H_MAX_21;
  const remaining_fraction = session.session_entropy / max_entropy;
  if (remaining_fraction < 0.20) {
    return { shouldStop: true, reason: `entropy floor: ${remaining_fraction *
100}.toFixed(1)}% remaining` };
  }

  // Marginal gain: best next question provides < 5% of remaining entropy
  if (session.question_queue.length > 0) {

```

```

const best_ewig = computeEWIG(session.question_queue[0], session);
const marginal_fraction = session.session_entropy > 0
  ? best_ewig / session.session_entropy
  : 0;
if (marginal_fraction < 0.05 && session.interaction_count >= 8) {
  return { shouldStop: true, reason: `marginal gain below threshold:
${(marginal_fraction * 100).toFixed(1)}%` };
}
}

return { shouldStop: false, reason: 'continue' };
}

```

7.4 How Multi-Axis NLP Shortens the Assessment

The key efficiency gain: when a rich voice response covers N axes, the question queue shrinks by N-1 axes (the asked axis would have been removed regardless). This cascading effect means:

- If the user answers 8 questions via cards (covering 8 axes directly + ~3 axes via cross-correlation imputation), 5 axes remain.
- If the user then provides 2 rich voice responses that each cover 3 axes (1 primary + 2 secondary confirmed), that covers 6 axes -- more than the 5 remaining.
- Total interactions: 10 (8 cards + 2 voice) to resolve all 16 axes.

The stopping criterion may fire even earlier if the cross-axis correlation matrix (Research 02) provides sufficient imputation for the remaining uncovered axes.

8. End-to-End Walkthrough

Scenario

Maria, a Michigan voter, starts the Civic Blueprint Assessment. Her ballot includes races touching 12 of the 16 axes. She answers 8 questions via cards, 2 via voice, and gets 6 axes resolved via secondary signals and imputation. She completes the full 16-axis profile in 10 interactions.

Step-by-step

Interaction 1: **econ_safetynet** (structured)

The sequencer selects **econ_safetynet** (high EWIG: it's ballot-relevant and has strong cross-domain correlations with healthcare and justice axes).

Maria sees 5 cards:

- "Broader safety net for everyone who needs it" (score 0.0)
- "Strong safety net with some eligibility requirements" (score 2.5)
- "Balanced approach" (score 5.0)
- "Safety net focused on those who truly can't help themselves" (score 7.5)
- "Minimal government safety net -- people should be self-reliant" (score 10.0)

She taps card 2 (score 2.5, confidence 0.80). Dwell time: 4200ms.

Session state after:

- **econ_safetynet**: covered, score 2.5, hybrid confidence 0.703
- Cross-axis propagation shifts posteriors on **econ_investment** ($\rho=0.65$), **health_coverage_model** ($\rho=0.55$), **justice_policing_accountability** ($\rho=0.40$)
- **consecutive_neutrals**: 0

Interaction 2: climate_ambition (structured)

Sequencer picks **climate_ambition** (high EWIG: strong correlation with **climate_energy_portfolio** at $\rho=0.70$, and ballot-relevant).

Maria taps card 1 (score 0.0 -- "Aggressive immediate climate action"). Confidence 0.90. Dwell: 1800ms.

Session state after:

- **climate_ambition**: covered, score 0.0, hybrid confidence 0.831
- Cross-axis: **climate_energy_portfolio** posterior concentrates heavily toward pole A ($\rho=0.70$). **climate_permitting** gets mild shift ($\rho=0.30$). **housing_transport_priority** shifts slightly ($\rho=0.40$).
- Session entropy drops significantly -- one strong pole selection with high correlations.

Interaction 3: justice_firearms (structured)

Sequencer picks **justice_firearms** (relatively independent axis -- low correlation with most non-justice axes, so it has high remaining entropy).

Maria taps card 4 (score 7.5 -- "Protect gun rights with minimal regulation"). Confidence 0.80. Dwell: 3100ms.

Session state after:

- **justice_firearms**: covered
- Cross-axis: mild shift on **justice_policing_accountability** ($\rho=0.50$) and **justice_sentencing_goals** ($\rho=0.45$)

Interaction 4: health_coverage_model (structured)

Even though cross-axis propagation from **econ_safetynet** already shifted this axis's posterior, the sequencer still asks -- the posterior entropy is reduced but not below the threshold, and this axis has high ballot relevance.

Maria taps card 2 (score 2.5 -- "Government option alongside private insurance"). Confidence 0.80.

Session state after:

- **health_coverage_model**: covered
- Cross-axis: strong propagation to **health_cost_control** ($\rho=0.60$), **health_public_health** ($\rho=0.50$)

Interaction 5: **housing_supply_zoning** (structured -- with T4 enhanced escape hatch)

This is a NUANCED_AXIS, so the "None of these fit" button is visually enhanced. Maria reads the cards but feels conflicted. She supports more housing construction in theory but has concerns about her own neighborhood. She taps "None of these fit -- tell us in your own words."

Trigger T1 fires. The NLP panel opens inline.

Interaction 5 (NLP): **housing_supply_zoning**

Maria speaks: "I want more housing built, especially near transit stops. I think we need to allow duplexes and small apartments in more neighborhoods, but I also think there should be design standards so new buildings fit in. And honestly, I'd rather see the government invest in public transit than keep widening highways."

Multi-axis extraction:

Signal	Axis	Direction	Confidence	Strength
Primary	housing_supply_zoning	3.0 (lean toward loosening zoning, with caveats)	0.70	primary
Secondary	housing_transport_priority	2.0 (strong public transit preference)	0.75	secondary
Secondary	housing_affordability_tools	3.0 (government-leaning but not extreme)	0.50	secondary
Spillover	climate_ambition	2.0 (transit preference correlates)	0.20	spillover

After classification and confidence discounting:

- **housing_supply_zoning**: hybrid confidence 0.62 (primary, full confidence)
- **housing_transport_priority**: hybrid confidence 0.54 (secondary, 85% of 0.64)
- **housing_affordability_tools**: hybrid confidence 0.36 (secondary, 85% of 0.42)
- **climate_ambition**: not shown (spillover), but posterior tightens slightly (already had strong signal from interaction 2)

Confirmation card shown:

We picked up a few things from your response:

[v] Zoning: Allow more housing types with design standards

[] Transit: Invest in public transit over highways

[] Affordability: Government should help with housing costs

Check the ones that sound right.

Maria checks "Transit" (she was very explicit about this). She leaves "Affordability" unchecked -- she said it but isn't sure it fully represents her view.

After confirmation:

- **housing_supply_zoning**: covered (primary, confirmed)
- **housing_transport_priority**: covered (secondary, user confirmed)
- **housing_affordability_tools**: partial (secondary, not confirmed -- applied as soft evidence only)
- 3 axes resolved in 1 interaction (asked about 1, confirmed 2 additional)

Interactions 6-8: Structured cards

The sequencer recomputes. The queue now skips **housing_transport_priority** (confirmed via voice). The remaining high-priority axes are:

- **econ_school_choice** (low cross-axis correlation, not imputable)
- **econ_tax_structure** (some soft signal from **econ_safetynet** correlation, but still above entropy threshold)
- **justice_sentencing_goals** (some signal from **justice_firearms** correlation)

Maria answers all three via cards with moderate or strong positions. No neutrals.

After interaction 8:

- 11 axes answered directly (8 structured + 3 from voice interaction 5)
- 5 axes remaining: **econ_investment**, **health_cost_control**, **health_public_health**, **housing_affordability_tools**, **climate_energy_portfolio**

But the posteriors on these 5 axes are no longer uniform:

- **econ_investment**: strong signal from **econ_safetynet** ($\rho=0.65$) and **econ_tax_structure** ($\rho=0.60$). Entropy: 1.4 bits (below the 1.5-bit axis threshold from Research 03).
- **health_cost_control**: signal from **health_coverage_model** ($\rho=0.60$). Entropy: 1.8 bits.
- **health_public_health**: signals from **health_coverage_model** ($\rho=0.50$) and **justice_policing_accountability** (cross-domain, $\rho=0.50$). Entropy: 1.6 bits.
- **housing_affordability_tools**: soft NLP signal + cross-axis from **housing_supply_zoning** ($\rho=0.25$). Entropy: 2.1 bits.
- **climate_energy_portfolio**: strong signal from **climate_ambition** ($\rho=0.70$). Entropy: 1.2 bits (well below threshold).

Interaction 9: **housing_affordability_tools** (structured)

This is the only remaining axis above the entropy threshold. The sequencer asks about it. Maria taps card 2 (government subsidies lean).

After interaction 9:

- **housing_affordability_tools**: covered
- Cross-axis: minor propagation

Stopping criterion check

After interaction 9:

- 12 axes directly answered
- 4 axes imputable: `econ_investment` (entropy 1.4), `health_cost_control` (1.7 after latest propagation), `health_public_health` (1.5), `climate_energy_portfolio` (1.2)
- Remaining weighted entropy fraction: 14.8% (below the 20% floor)

Session stops. The sequencer reports: "entropy floor reached after 9 interactions."

Wait -- let us also account for interaction 5 being one NLP interaction. Total: 8 structured + 1 NLP + confirmation = 9 interactions producing 12 directly answered axes + 4 imputed axes = 16 total.

But Maria's voice response was so rich that the system actually resolved 3 axes in that single turn. Without the multi-axis extraction, she would have needed 2 more structured questions (for `housing_transport_priority` and `housing_affordability_tools`), plus the question queue might not have reached the stopping criterion until interaction 11 or 12.

Interaction 10: Summary and confirmation

The system shows Maria a summary of her complete profile:

Your Civic Blueprint (16 axes)

Economic Policy:

Safety Net: Broader support (2.5/10)	[card]
Public Investment: More investment (2.8/10)	[imputed from related answers]
School Choice: Traditional public (2.0/10)	[card]
Tax Structure: Progressive (2.5/10)	[card]

Healthcare:

Coverage Model: Public option (2.5/10)	[card]
Cost Control: Government oversight (3.1/10)	[imputed]
Public Health: Balanced (4.2/10)	[imputed]

Housing:

Zoning: Loosen with standards (3.0/10)	[voice]
Affordability: Government subsidies (2.5/10)	[card]
Transit: Public transit priority (2.0/10)	[voice - confirmed]

Justice:

Policing: Moderate reform (4.0/10)	[imputed]
Sentencing: Rehabilitation lean (3.5/10)	[card]
Firearms: Protect gun rights (7.5/10)	[card]

Climate:

Ambition: Aggressive action (0.0/10)	[card]
Energy: Renewables priority (1.8/10)	[imputed]
Permitting: Slightly faster (4.5/10)	[imputed, low confidence]

9 direct answers + 4 imputed = 16 axes complete

Assessment quality: HIGH (weighted confidence 0.72)

Maria can tap any imputed axis to override it with a card selection or voice input. If she does nothing, the profile is finalized and used for ballot matching.

Walkthrough Summary

Step	Axis	Modality	Axes resolved	Cumulative
1	econ_safetynet	Card	1	1
2	climate_ambition	Card	1	2
3	justice_firearms	Card	1	3
4	health_coverage_model	Card	1	4
5	housing_supply_zoning	Voice	3 (1 primary + 2 secondary confirmed)	7
6	econ_school_choice	Card	1	8
7	econ_tax_structure	Card	1	9
8	justice_sentencing_goals	Card	1	10
9	housing_affordability_tools	Card	1	11
--	(imputation)	Cross-axis	5	16

Total user interactions: 9 (8 cards + 1 voice with confirmation) **Total axes resolved: 16** (11 direct + 5 imputed) **Savings vs. full 16-question battery: 7 questions (44% reduction)**

The voice interaction contributed the most savings: 3 axes in 1 turn (vs. 3 separate card questions), plus the soft evidence it deposited on other axes accelerated the stopping criterion.

9. Implementation Notes

9.1 File Structure

File	Purpose
src/types/hybridAssessment.ts	All interfaces from Section 4
src/lib/hybridSession.ts	Session initialization, state management, question queue
src/lib/modalityTriggers.ts	Trigger evaluation (Section 2)
src/lib/multiAxisExtraction.ts	Signal classification, confirmation processing (Section 5)
src/lib/conflictResolution.ts	Merge and contradiction handling (Section 6)
src/components/assessment/CardQuestion.tsx	Structured card UI with escape hatch
src/components/assessment/NlpPanel.tsx	Voice/text input panel (inline)

File	Purpose
<code>src/components/assessment/ConfirmationCard.tsx</code>	Multi-axis confirmation UI
<code>src/components/assessment/ProfileSummary.tsx</code>	Final profile display with override option
<code>src/app/api/assessment/extract/route.ts</code>	LLM extraction endpoint (multi-axis)

9.2 Constants

```
// Entropy confidence blending (from Research 06)
const ALPHA_STRUCTURED = 0.35;
const ALPHA_NLP = 0.55;

// 21-bin posterior (from Research 06)
const N_BINS = 21;
const H_MAX_21 = Math.log2(N_BINS); // 4.392 bits

// Stopping thresholds
const MIN_INTERACTIONS = 5;
const MAX_INTERACTIONS = 20;
const ENTROPY_FLOOR_FRACTION = 0.20;
const MARGINAL_GAIN_THRESHOLD = 0.05;

// Confirmation gate
const STRUCTURED_CONFIRMATION_GATE = 0.60;
const NLP_CONFIRMATION_GATE = 0.50;

// Contradiction threshold
const CONTRADICTION_SCORE_GAP = 3.0;

// Strength discounts
const SECONDARY_DISCOUNT = 0.85;
const SPILLOVER_DISCOUNT = 0.60;
```

9.3 Posterior Representation Reconciliation

Research 02 uses a 5-point discrete posterior (matching the 5 card options). Research 03 uses an 11-bin posterior. Research 06 uses a 21-bin posterior. The hybrid session standardizes on the **21-bin representation** because:

1. It has sufficient resolution for both structured and NLP signals.
2. Structured card selections map cleanly to bins 0, 5, 10, 15, 20 (the 0.0, 2.5, 5.0, 7.5, 10.0 positions).
3. NLP signals can target any bin, not just the 5 card positions.
4. The entropy calculations from Research 06 are already calibrated for 21 bins.

Conversion from 5-point to 21-bin is lossless (place mass at the matching bins). Conversion from 11-bin to 21-bin is interpolation (spread each bin's mass across the two nearest 21-bin neighbors).

9.4 Latency Budget

The critical path for an NLP interaction is:

1. Speech-to-text (if voice): ~500ms
2. LLM extraction (Template A, two-pass): ~1500ms
3. Signal classification + posterior update: ~5ms
4. Confirmation card rendering: ~50ms

Total: ~2050ms for voice, ~1550ms for text.

The structured path is instant (no LLM call). The confirmation card adds no latency because it's generated client-side from the already-computed signals.

To keep the NLP path feeling responsive, show the confirmation card progressively: display the primary signal immediately after extraction, then animate in secondary signals as they're classified.

9.5 Bias Safeguards

1. **The escape hatch must be equally available on all axes.** Do not suppress it on axes where the NLP path might expose user views the system "prefers."
2. **The multi-axis extraction must not hallucinate signals.** The LLM extraction prompt explicitly states: "Only include axes where the user DIRECTLY stated a position." The `isExplicitMention()` check provides a second gate.
3. **Secondary signals default to unchecked.** The user must opt in. This prevents the system from "putting words in the user's mouth."
4. **Imputed axes are clearly labeled** in the profile summary. The user can always override.
5. **The stopping criterion is neutral.** It measures information content, not ideological completeness. A libertarian who gives clear positions on 8 axes and neutral on 8 axes will stop at the entropy floor, not be pushed to answer more.